CSC343HY Tutorial 4

Aggregation, Subqueries, and Temporary Tables

Last week you learned to read a single table with `SELECT`, `FROM`, and `WHERE`: pick some columns, keep some rows. Every query you wrote returned data that was already sitting in the table, one output row per input row. This week we break that one-to-one correspondence in three different ways. **Aggregation** collapses many rows into one summary value. **Subqueries** let a query use the result of another query as one of its ingredients. And **temporary tables** let you save an intermediate result under a name and reuse it, inside or outside a later query. Each is a tool for the same underlying problem: questions whose answers do not fit neatly into "filter the rows of one table."

1. Getting Set Up

As before, you can either run today's queries against the lab database or read along on paper and predict each result. If you want to run them, the steps are the same as the last two weeks.

Connecting

```
psql -h mcsdb.utm.utoronto.ca -U yourutorid -d yourutorid_343
```

Off campus, connect to the UofT VPN first (`general.vpn.utoronto.ca`, group `UofT Default`).

Loading the sample data

We continue with the **Community Library** schema. If the schema and the Tutorial 2 sample data are still loaded, you are ready. If not, reload the DDL and then download `Tutorial3-SQL.dml` from the course site and run it.

```
psql -h mcsdb.utm.utoronto.ca -U yourutorid -d yourutorid_343 -f
    Tutorial3-SQL.dml
```

The Tutorial 3 data file adds a `loan` table on top of the `book_copy`, `member`, and `branch` rows you already have. A `loan` records that a member borrowed a particular copy: it has a `loan_id`, a `book_copy_id`, a `member_id`, a `loan_date`, and a `fine_amount` (the late fee owed on that loan, possibly `0`).

> Re-running the data file may produce duplicate-key errors. That is expected and harmless: psql skips rows that already exist and continues. For a clean slate, `TRUNCATE` the tables in dependency order and reload.

Our running tables

Today's examples use `book_copy` (unchanged from last week) and the new `loan` table. After loading the sample data, `loan` looks roughly like this:

loan_id	book_copy_id	member_id	loan_date	fine_amount
1	103	7	2024-01-09	0.00
2	101	7	2024-01-22	4.50
3	104	12	2024-02-03	0.00
4	102	7	2024-02-15	1.25
5	105	4	2024-03-01	12.00
6	103	12	2024-03-19	0.00
7	107	4	2024-04-02	3.75
8	101	19	2024-04-28	0.00
9	108	12	2024-05-11	0.00
10	106	4	2024-05-30	8.00

Recall the `book_copy` rows from last week (`book_copy_id` 101–108, with a `title`, `condition`, `acquisition_date`, `owning_branch_name`, and `replacement_cost`). Run `\d+ loan` in psql for the full schema.

2. Aggregation

So far every query returned one output row for each input row that survived the `WHERE` clause. An **aggregate function** does the opposite: it takes a whole set of values and computes a *single* value from it. "How many loans are there?" is not a question about any one loan, it is a question about the set of all of them.

The five aggregate functions

Function	What it computes
`COUNT`	The number of rows. `COUNT(*)` counts every row; `COUNT(col)` counts rows where `col` is not null.
`SUM`	The total of all non-null values in a numeric column.
`AVG`	The arithmetic mean of all non-null values in a numeric column.
`MAX`	The largest value (works on numbers, strings, and dates).
`MIN`	The smallest value.

A query that uses an aggregate in its `SELECT` returns exactly one row:

```
SELECT COUNT(*)
FROM   loan;
```

Result: a single row, single column, value `10`. Aggregates also respect `WHERE`, which runs first and decides which rows the aggregate even sees:

```
SELECT COUNT(*), SUM(fine_amount), AVG(fine_amount)
FROM   loan
WHERE  fine_amount > 0;
```

Here `WHERE` keeps only the five loans that actually carry a fine, and the three aggregates summarise just those five rows.

> *`COUNT(*)` and `COUNT(col)` are not the same. `COUNT(*)` counts rows, full stop. `COUNT(col)` counts rows*

where `col` *is not null, so it can be smaller.* `SUM` *and* `AVG` *also ignore nulls, which means* `AVG` *is the sum of non-null values divided by the* count of non-null values*, not by the total row count.*

Naming the output column with `AS`

By default an aggregated column gets an unhelpful name like `count` or `?column?`. Use `AS` to give it a readable name:

```
SELECT COUNT(*)         AS num_loans,
       SUM(fine_amount) AS total_fines
FROM   loan;
```

Grouping with `GROUP BY`

A plain aggregate collapses the whole table into one row. Often you want one summary row *per category*: total fines per member, number of loans per branch. That is what `GROUP BY` is for. It partitions the rows into groups that share a value, then computes the aggregate once per group.

```
SELECT member_id,
       COUNT(*)          AS num_loans,
       SUM(fine_amount) AS total_fines
FROM   loan
GROUP BY member_id;
```

This returns one row per distinct `member_id`: member `7` with 3 loans, member `12` with 4, member `4` with 3, member `19` with 1.

The golden rule of `GROUP BY`. Every column in your `SELECT` must either be (a) one of the columns you grouped by, or (b) wrapped in an aggregate function. A column that is neither has no single value for the group, so PostgreSQL rejects the query. If you find yourself wanting to select such a column, you probably need to either group by it too or aggregate it.

Filtering groups with `HAVING`

`WHERE` filters *rows* before grouping. To filter *groups* after the aggregate is computed, e.g. "only members who owe more than $5 in total," you need `HAVING`:

```
SELECT member_id, SUM(fine_amount) AS total_fines
FROM   loan
GROUP BY member_id
HAVING SUM(fine_amount) > 5;
```

`WHERE` vs. `HAVING`: `WHERE` cannot mention an aggregate (it runs before any grouping happens), and `HAVING` is the clause that can. A useful mental order is `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT`. Read your query in that order and it is clear which filter sees what.

3. Subqueries

A **subquery** is a query nested inside another query. It runs first, and its result is used by the outer query. Subqueries matter because some questions are inherently two-step: "which copies cost more than the average?" first needs the average, then needs a comparison against it. You cannot put an

aggregate directly in a `WHERE` clause, but you can put it in a subquery and compare against that.

Scalar subqueries: a subquery that returns one value

If a subquery returns exactly one row and one column, it behaves like a single value and can sit anywhere a value can:

```
SELECT title, replacement_cost
FROM   book_copy
WHERE  replacement_cost > (SELECT AVG(replacement_cost)
                           FROM book_copy);
```

The inner query computes one number, the average cost. The outer query then compares each copy against it. This is the standard fix for "I want to filter by an aggregate": move the aggregate into a subquery.

Subqueries with `IN`: a subquery that returns a column

A subquery can also return a whole column of values. Pair it with `IN` to test membership:

```
SELECT title
FROM   book_copy
WHERE  book_copy_id IN (SELECT book_copy_id
                        FROM loan
                        WHERE fine_amount > 0);
```

The inner query produces the set of copy ids that have ever been fined; the outer query returns the titles of exactly those copies. You can negate it with `NOT IN` to get copies that have never been fined.

Testing existence with `EXISTS`

`EXISTS` takes a subquery and is true if the subquery returns *at least one row*. It is often used with a *correlated* subquery, one that refers to a column from the outer query:

```
SELECT title
FROM   book_copy bc
WHERE  EXISTS (SELECT 1
               FROM loan l
               WHERE l.book_copy_id = bc.book_copy_id);
```

For each copy `bc`, the inner query looks for a loan of that copy. If one exists, the copy is kept. This returns every copy that has been loaned at least once. The `SELECT 1` is idiomatic: with `EXISTS` only the presence of rows matters, never their contents.

Which subquery shape do I need? If you need a *single value* (to compare with `=`, `<`, `>`), write a scalar subquery. If you need a *set of values* to test membership against, use `IN`. If you only need to know *whether any matching row exists*, use `EXISTS`. Many questions can be answered by more than one of these; pick the one that reads most clearly.

A scalar subquery that accidentally returns more than one row is a runtime error. If `WHERE cost > (SELECT ...)` complains that the subquery returned multiple rows, your inner query is not as specific as you thought, add a condition or an aggregate so it yields exactly one value.

4. Temporary Tables

When a query gets complicated, it helps to compute an intermediate result once, give it a name, and reuse it. A **temporary table** does exactly that. It is a real table you can `INSERT` into and `SELECT` from like any other, except it is created with `TEMPORARY` and exists only for the duration of your current session. When you disconnect, it disappears automatically; no cleanup needed.

Creating and populating one

You can create a temporary table and fill it in one step from a query:

```
CREATE TEMPORARY TABLE member_fines AS
SELECT member_id, SUM(fine_amount) AS total_fines
FROM   loan
GROUP BY member_id;
```

`member_fines` now holds one row per member with their fine total. From here on you can treat it as an ordinary table:

```
SELECT *
FROM   member_fines
WHERE  total_fines > 5;
```

Using a temporary table as a "variable"

This is where temporary tables earn their keep. A result you computed once can be referred to again, both *outside* a query (run another query against it later) and *inside* one (join it, or use it in a subquery). For example, to find which members owe more than the average member fine:

```
SELECT member_id, total_fines
FROM   member_fines
WHERE  total_fines > (SELECT AVG(total_fines)
                      FROM member_fines);
```

Here `member_fines` plays the role of a variable: computed once, then used twice in the same query, once in the `FROM` and once inside the subquery. Without it you would have to write the grouping query out twice and hope you typed it identically both times.

Why bother, when a subquery would also work? Two reasons. First, *clarity*: a long query split into a couple of named temporary tables is far easier to read and debug than one deeply nested monster. Second, *reuse*: if the same intermediate result is needed in several places, computing it once into a temporary table avoids repeating, and re-deriving, it. The trade-off is that a temporary table is a separate statement and persists for the session, whereas a subquery is self-contained.

A temporary table lives only in your session. Another student connected to the same database cannot see your `member_fines`, and it vanishes when you disconnect. If you re-run a `CREATE TEMPORARY TABLE` for a name that already exists in your session, you will get an error, drop it first with `DROP TABLE`, or use `CREATE TEMPORARY TABLE IF NOT EXISTS`.

5. Exercises

Work through these with your neighbour. Predict the result before running each query, then run it and check. Exercises use the `loan` and `book_copy` tables from the front of the handout.

Aggregation

1. Write a query that returns the total number of loans in the `loan` table.

2. Write a query that returns the largest single `fine_amount` and the smallest single `fine_amount` across all loans. Give both output columns readable names with `AS`.

3. Write a query that returns the average `fine_amount` over only those loans that carry a fine (`fine_amount` greater than 0). Then write it again over *all* loans. Explain to your neighbour why the two answers differ.

4. Write a query that returns, for each `member_id`, how many loans that member has. Use `GROUP BY`.

5. Write a query that returns, for each `member_id`, their total fines, but only for members whose total fines exceed $5. Use `GROUP BY` and `HAVING`.

Subqueries

6. Write a query that returns the `title` and `replacement_cost` of every copy whose replacement cost is below the average replacement cost. Use a scalar subquery.

7. Write a query that returns the `title` of every copy that has been loaned at least once. Do it once with `IN` and a subquery over `loan`, then again with `EXISTS` and a correlated subquery.

8. Write a query that returns the `title` of every copy that has *never* been loaned. Hint: `NOT IN`.

9. **Trickier.** Write a query that returns the `title` of every copy that has been loaned by member `4`. Think about which table tells you the copy ids member `4` borrowed, and which table maps those ids to titles.

Temporary tables

10. Create a temporary table `loan_counts` holding one row per `member_id` with a column `num_loans` giving that member's loan count. Then write a separate query against `loan_counts` that returns only the members with more than 2 loans.

11. Using the `member_fines` temporary table from Section 4 (recreate it if your session was reset), write a query that returns the `member_id` of every member whose total fines are strictly greater than the average total fine across all members.

12. **Trickier.** Create a temporary table `copy_loan_counts` with one row per `book_copy_id` giving the number of times that copy was loaned. Then write a query that joins it back to `book_copy` to return each `title` alongside its loan count, including copies loaned zero times. (Hint: think about which join keeps the unloaned copies, and how a count behaves over them.)

6. Common Pitfalls

A checklist of the mistakes that come up most often once aggregation and subqueries enter the picture. If a query misbehaves, scan here first.

1. Selecting a non-aggregated, non-grouped column alongside an aggregate. Every `SELECT` column must be grouped by or aggregated.
2. Putting an aggregate in `WHERE`. `WHERE` runs before grouping; to filter on an aggregate, use `HAVING`.
3. Forgetting that `SUM`, `AVG`, and `COUNT(col)` silently ignore nulls, so `AVG` may not divide by the row count you expect.
4. Confusing `COUNT(*)` with `COUNT(col)`. The first counts rows, the second counts non-null values of that column.
5. Writing a scalar subquery that can return more than one row. If used with `=`, `<`, or `>`, that is a runtime error, make it specific or aggregate it.
6. Using `NOT IN` with a subquery whose result contains a `NULL`. This can make the whole condition evaluate to nothing useful; prefer `NOT EXISTS` when nulls are possible.
7. Re-running `CREATE TEMPORARY TABLE` for a name that already exists in the session. Drop it first, or use `IF NOT EXISTS`.
8. Expecting a temporary table to still be there in a new session. It is not, it is gone the moment you disconnect.

Before You Leave

Today's priority is being able to move beyond one-output-row-per-input-row: collapse rows with aggregation, feed one query's result into another with subqueries, and stash an intermediate result in a temporary table so you can reuse it. These three ideas combine constantly, a temporary table holding a grouped aggregate, queried with a subquery, is an extremely common pattern. If you finished early, try rewriting one of the subquery exercises using a temporary table instead, and decide which version you find clearer.

Checklist

1. I can use `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX` to collapse a table into a single summary row.
2. I can use `GROUP BY` to get one summary row per category, and I know the rule about which columns may appear in `SELECT`.
3. I know the difference between `WHERE` and `HAVING` and when each one runs.
4. I can write a scalar subquery and use it inside a `WHERE` comparison.
5. I can use a subquery with `IN` and with `EXISTS`, and I know what shape of result each one expects.
6. I can create a temporary table from a query and then both query it again later and use it inside another query.

7. I understand that a temporary table lasts only for my session.